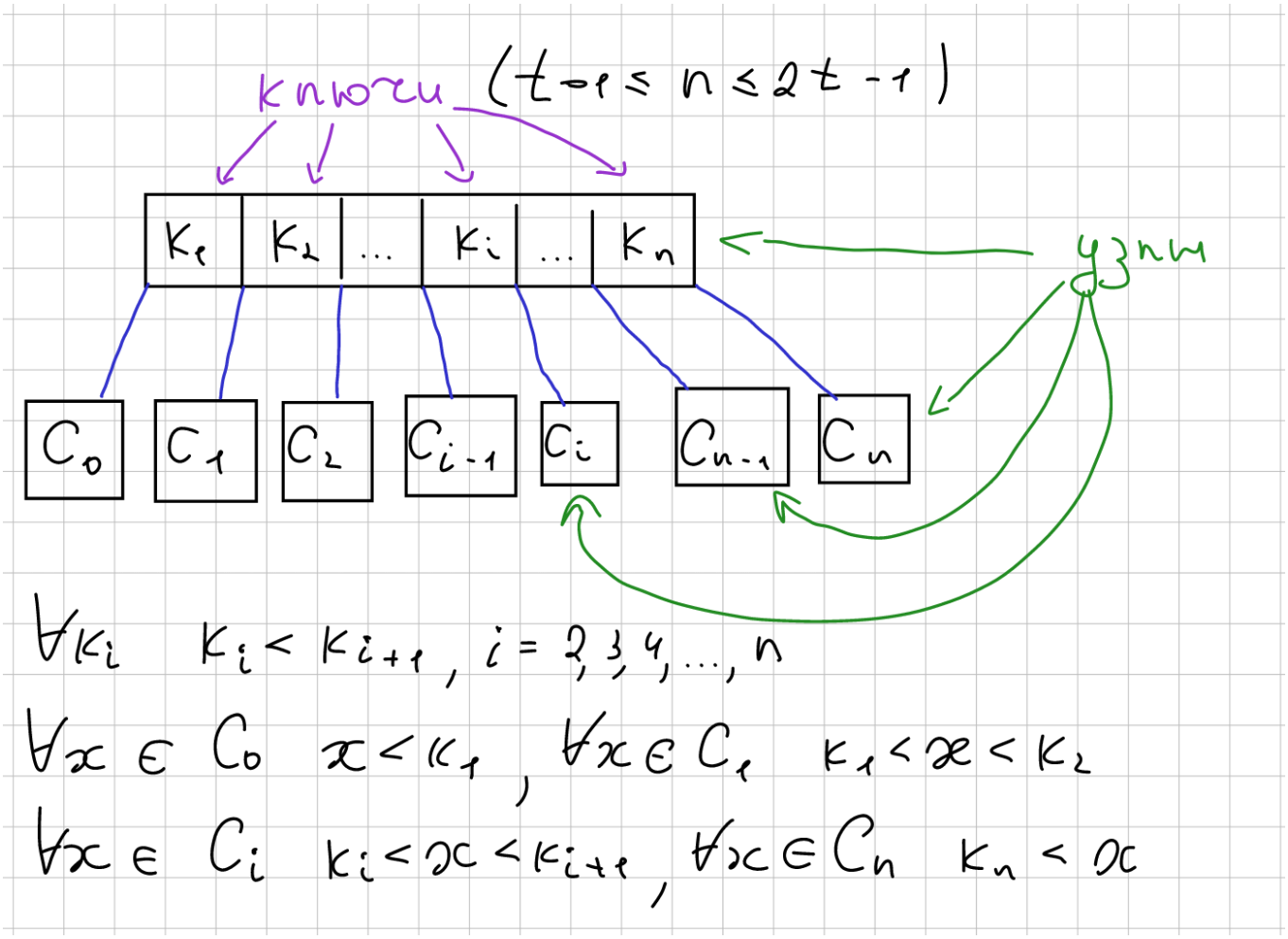
Зачем нужно B-дерево

B-дерево применяется в случаях, когда дерево очень большое и хранится кусками не в *оперативной памяти*, а разбито на блоки, хранящиеся в "файлах" внутри файловой системы жёстких дисков/SSD. В таком случае, мы знаем, куда обращаться в память, чтобы просто прочитать и загрузить в оперативку только нужный нам блок. В самой же оперативной памяти **всегда** хранится `root` этого дерева. В качестве упрощения, будем считать, что загрузка необходимого блока в память (прочтение и запись) осуществляются за константное время.

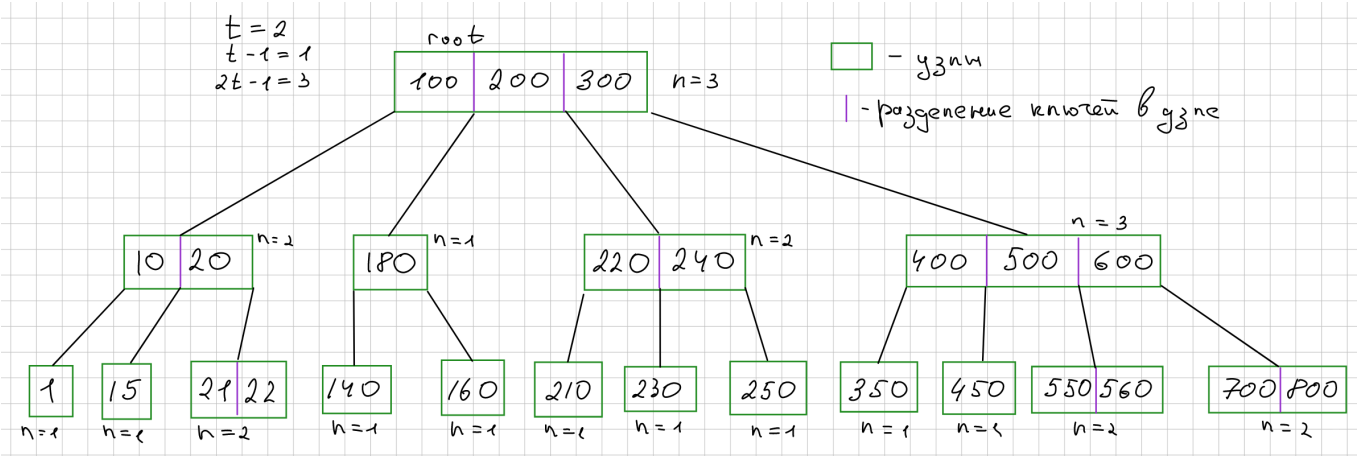
Основное определение B-дерева

B-деревом называется идеально сбалансированное дерево, обладающее следующими свойствами:

1. Каждый узел должен содержать в себе информацию о: ключи $k_i, i = 1, 2, 3, \ldots, n$ (отсортированные в порядке возрастания), количество этих ключей n, является ли узел листом
2. Каждый внутренний узел дерева содержит $n + 1$ указатель на дочерние узлы $c_0, c_1, \ldots, c_i, \ldots, c_n$. **ИСКЛЮЧЕНИЕ:** листья указывают на `nullptr`, больше в данном типе дерева `nullptr` нигде не используется!
3. Между двумя ключами k_i и k_{i+1} идёт указатель на c_i, в котором хранятся **ВСЕ** ключи, удовлетворяющие условию $k_i < c_i < k_{i+1}$.
4. **ВСЕ** листья расположены на одной высоте
5. Имеется нижняя и верхняя граница, которые задают минимальное и максимальное количество ключей в одном узле. Для этого определим переменную t. Тогда минимальное количество ключей в узле будет $t - 1$, а максимальное $2t - 1$ или количество ключей в узле задаётся неравенством $t - 1 \leq n \leq 2t - 1$. Данное неравенство верно для *всех* узлов, кроме **корня**. Для корня неравенство имеет вид: $1 \leq n \leq 2t - 1$.



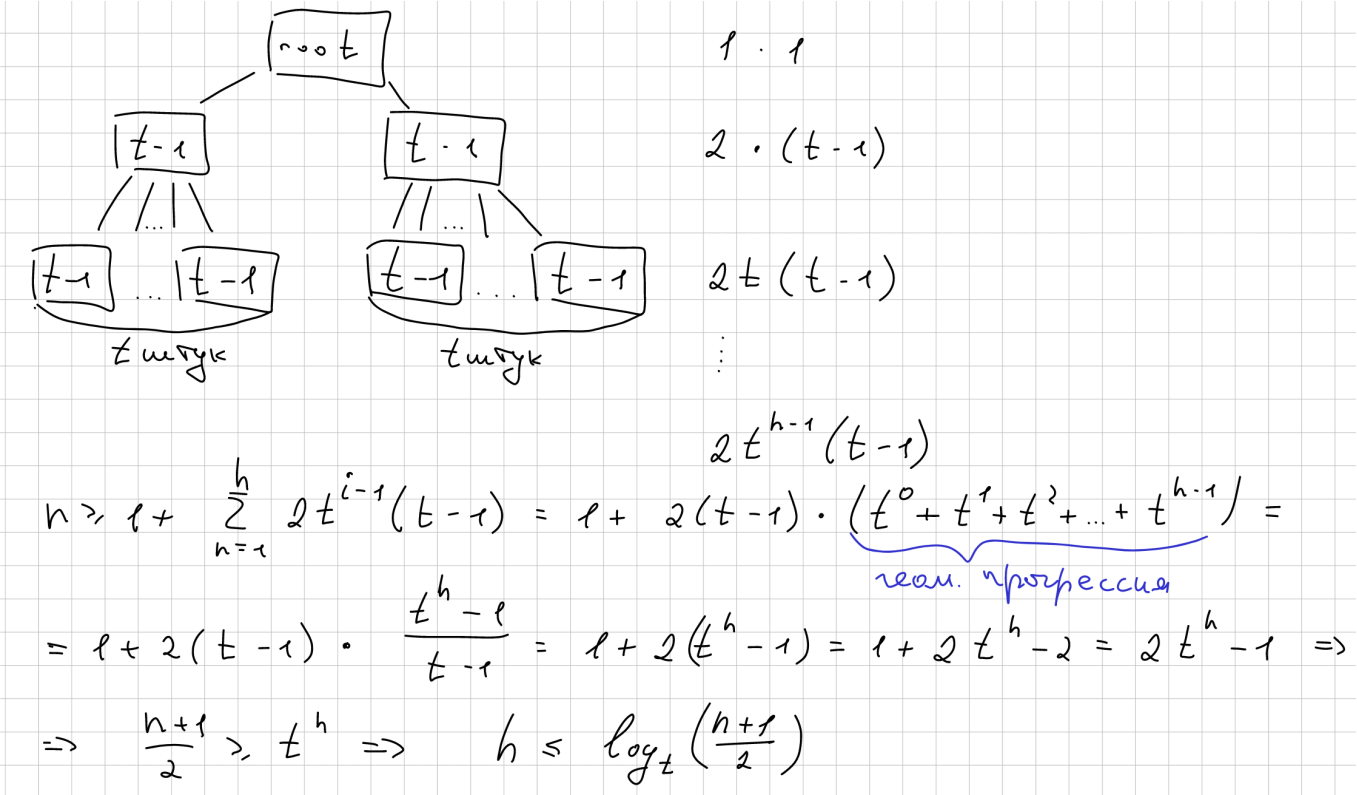
Пример:



Высота B-дерева

Теорема: Высота B-дерева `T` с $n \geq 1$ узлами и $t \leq 2$ не превышает $\log_t \frac{(n+1)}{2}$

Доказательство: Пусть h - высота дерева. Корень содержит минимум 1 ключ, а все остальные узлы содержат $t-1$ ключа. Тогда на глубине 1 минимум 2 узла, на глубине 2 - $2t$ узлов, на глубине 3 - $2t^2$ узлов и тд. Получается, что на глубине h будет минимум $2t^{h-1}$ узел. Тогда количество ключей удовлетворяет неравенству:
$n \geq 1 + \sum_{i=1}^{h}(t-1) \cdot 2t^{i-1} = 2t^h - 1, \quad => \quad h \leq \log_t \frac{n+1}{2}.$



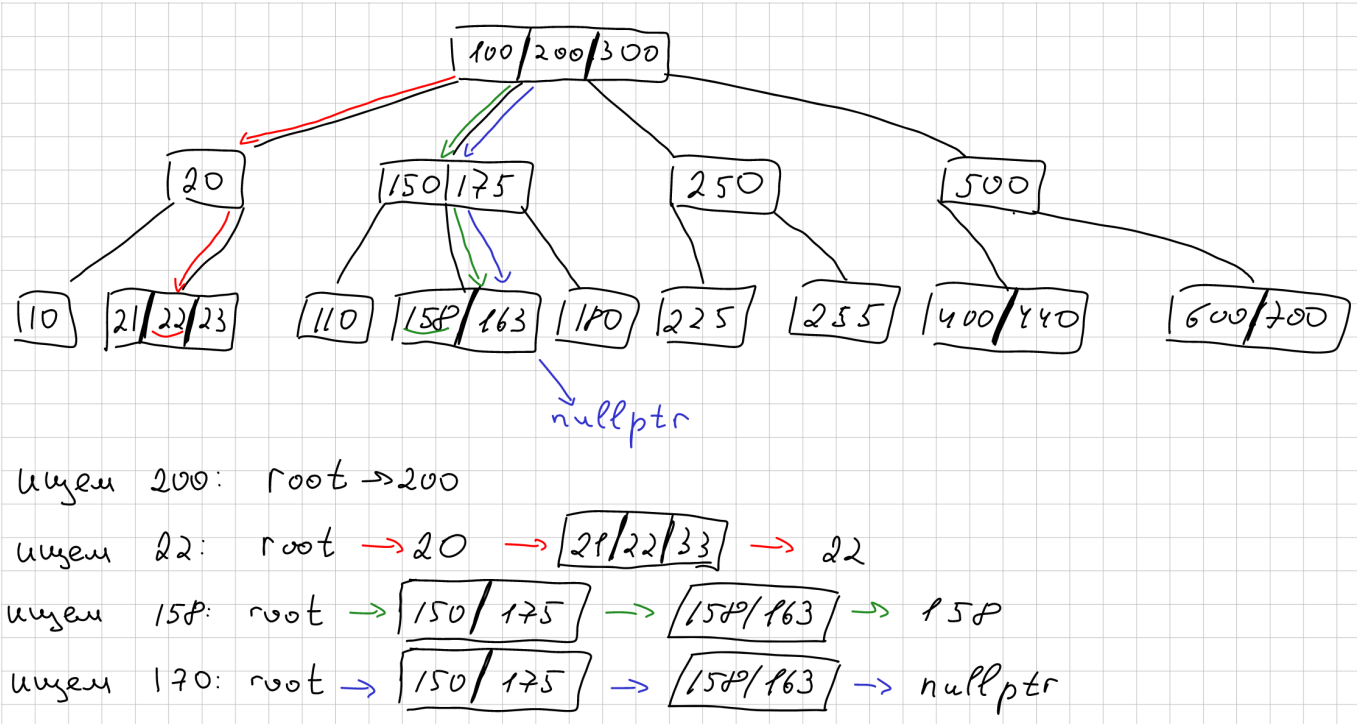
Операции над B-деревьями

Поиск в B-дереве

Допустим мы ищем key в таком узле: | k_1 | k_2 | ... | k_i | ... | k_n |
Возможные ситуации:

1. $k_i = key$ - значит, что ключ находится в этом узле.
2. $\forall i \in [1, 2, \dots, n] : k_i \neq key$:
   - $key < k_1$ - $search(c_0, key)$ - частный случай
   - $key > k_n$ - $search(c_n, key)$ - частный случай
   - $k_i < key < k_{i+1}$ - $search(c_i, key)$ - общий случай
     После каждого случая переходим в ноду, где начинаем проверку с п.1

Пример:



Сложность: $O(h \log t)$, h - высота дерева, $\log t$ - самый быстрый способ найти ключ/промежуток, где продолжить поиск через **бинарный поиск**. Данная оценка достигается тогда и только тогда, когда **ВСЁ** дерево загружено в оперативную память!

Вставка в B-дереве

> *Вставка всегда выполняется в лист!*

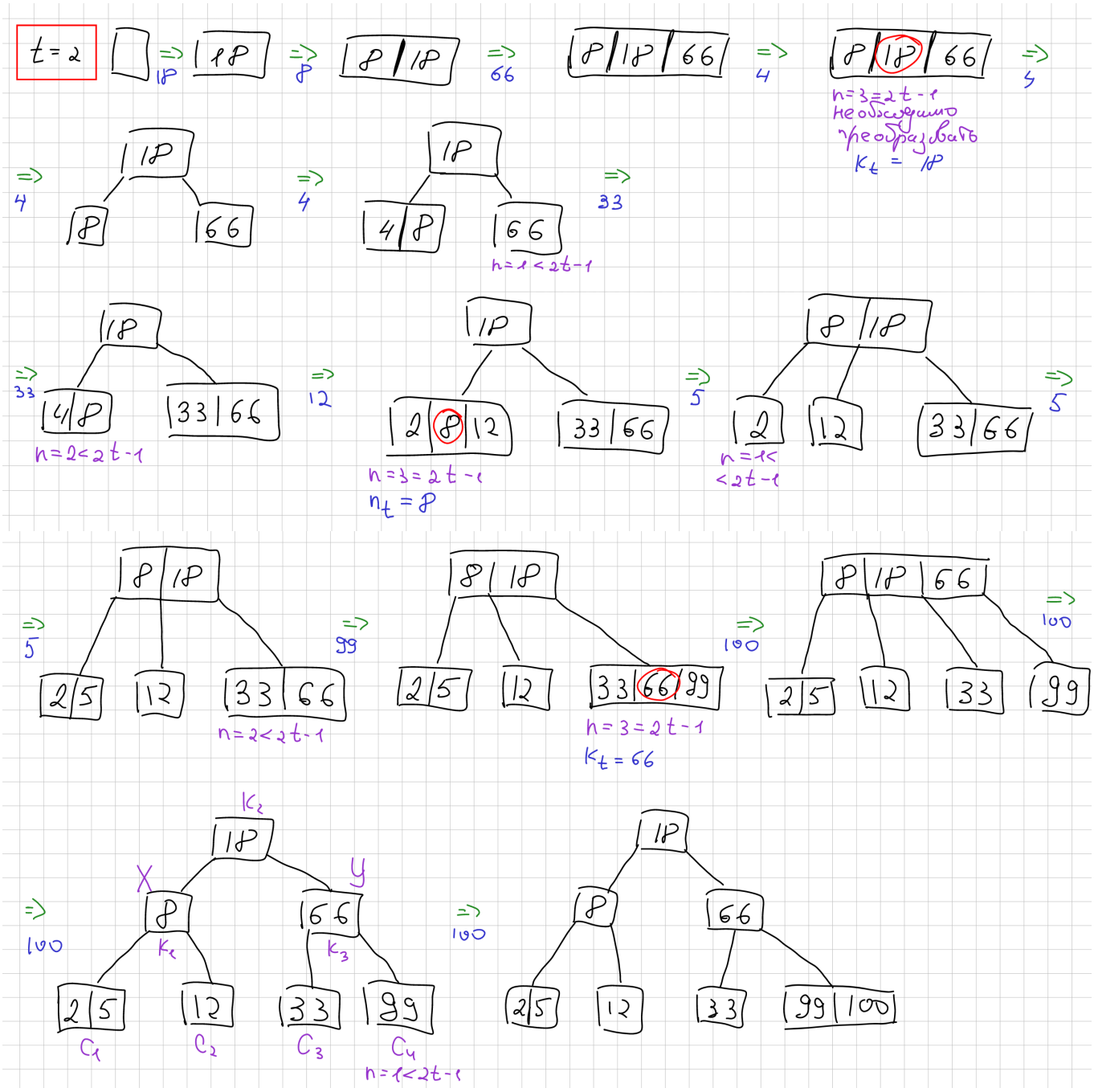
Перед вставкой ключа необходимо выполнить поиск до места, где он предположительно должен располагаться. Тогда возможны следующие ситуации:

1. Если количество ключей в узле $n < 2t - 1$, ТО можно спокойно вставлять ключ. В таком случае в узле станет $n \leq 2t - 1$ ключей, что удовлетворяет условию.
2. Если ключей в узле $n = 2t - 1$, то выделяем в этом узле ключ k_t как опорный. Далее ключи $k_1, k_2, \ldots, k_{t-1}$ выносим в отдельный узел X, а ключи $k_{t+1}, k_{t+2}, \ldots, k_{2t-1}$ в узел Y, указатели на узлы $c_0, c_1, \ldots, c_{k-1}$ достаются узлу X, указатели на узлы $c_k, c_{k+1}, \ldots, c_n$ достаются узлу Y. Сам опорный ключ k_t вставляем в его родителя между ключами, которые разделяли разрезаемый узел. "Левым" потомком ключа k_t становится узел X, "правым" - узел Y. Смотрим в какой из двух новых узлов необходимо вставлять элемент, спускаемся в него и согласно п.1 вставляем.

ВАЖНО: Когда пытаемся вставить ключ в узел выше, необходимо убедится, что в нём количество узлов $m < 2t - 1$. В ином случае нужно запускать такую же проверку для этого узла.
Также важно: Если предок это *корень* с $2t - 1$ ключами, то в нём можно вынести этот

элемент на уровень выше. В таком и только таком случае высота B-дерева увеличится на 1 в процессе вставки.

Пример:

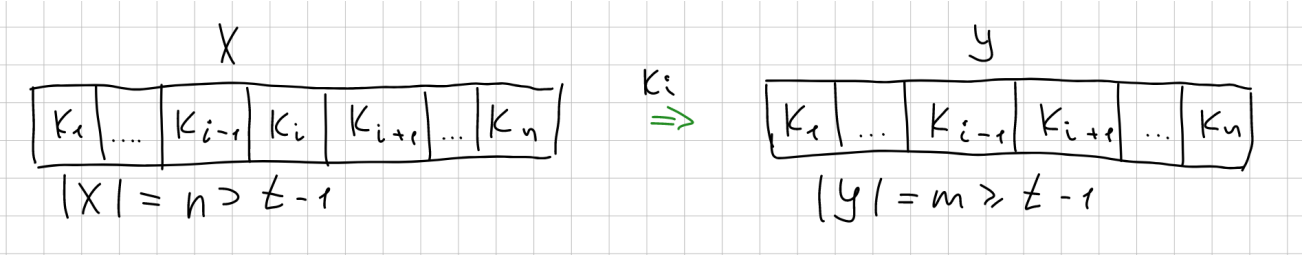


Сложность: $O(h \log t)$, комментарий аналогичен поиску

Удаление из B-дерева

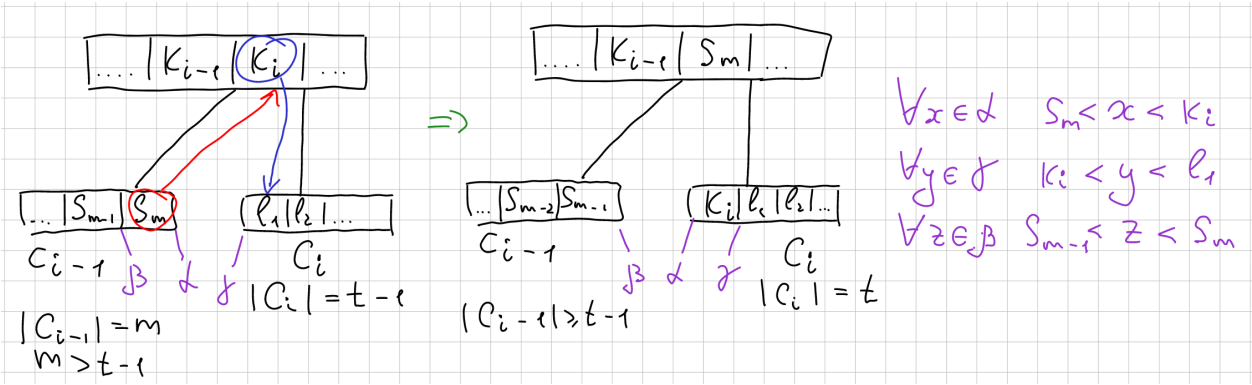
При удалении необходимо найти узел, в котором находится ключ. Далее рассматриваются случаи:

1. Если узел - лист и количество ключей $n > t - 1$, то можно спокойно удалять. Тогда в узле станет $n \geq t - 1$ ключей.

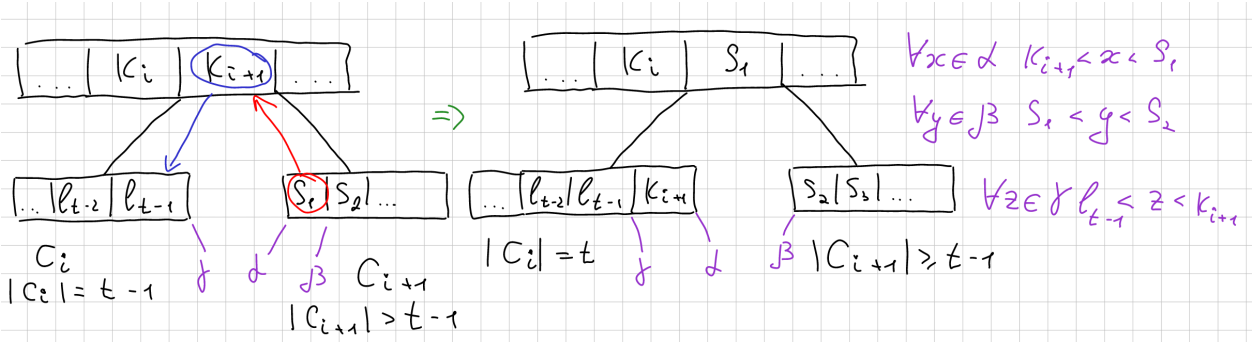


2. Если узел - лист и количество ключей $n = t - 1$, то просто удалить нельзя, т.к. в таком случае будет нарушено свойство ($n \geq t - 1$). Допустим мы находимся в узле c_i. Тогда на выбор обращаемся к одному из братьев c_{i-1}, c_{i+1}. В зависимости от того, какого брата выбрали делаем:

   - Если выбрали левого брата c_{i-1} и $|c_{i-1}| > t - 1$, то берём его максимальный (последний) ключ и записываем вместо k_i у нашего предка. А сам ключ k_i дописываем самым первым в узел c_i. Тогда $|c_i| > t - 1$ и согласно п.1 из него можно удалить любой элемент.

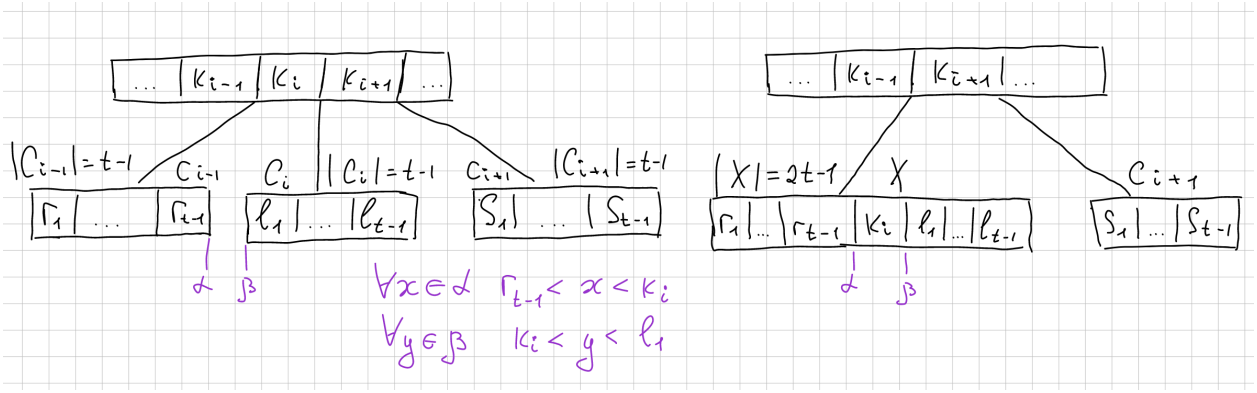


   - Если выбрали правого брата c_{i+1} и $|c_{i+1}| > t - 1$, то берём его минимальный (первый) ключ и записываем вместо k_{i+1} у нашего предка. А сам ключ k_{i+1} дописываем самым последним в узел c_i. Тогда $|c_i| > t - 1$ и согласно п.1 из него можно удалить любой элемент.



   - Если у обоих братьев количество ключей $|c_{i-1}| = |c_{i+1}| = t - 1$. В таком случае выбираем любого соседнего брата. Берём в предке ключ, который разделяет c_i и выбранного брата. Объединяем братьев, где в середине будет выбранный ключ. Таким образом получаем из двух узлов один, мощность которого составляет

$2t - 1 > t - 1$ ключ. Согласно п.1 из него можно удалить любой элемент



3. Если узел с нужным ключом - не лист, тогда у нас есть узлы как минимум ещё на один уровень ниже. В таком случае применяется следующая стратегия:
    1. Идём в правого ребёнка, если в нём больше чем $t - 1$ ключ. Далее находим минимальный ключ (самый левый) в его поддереве, запоминаем значение и пытаемся удалить этот ключ. Возвращаемся к удаляемому изначально ключу и перезаписываем его значение.
    2. Идём в левого ребёнка, если в нём больше чем $t - 1$ ключ. Далее находим максимальный ключ (самый правый) в его поддереве, запоминаем значение и пытаемся удалить этот ключ. Возвращаемся к удаляемому изначально ключу и перезаписываем его значение.
    3. Если оба потомка имеют по $t - 1$ ключ, то объединяем их с удаляемым ключом-разделителем в один узел, затирая его значение.

БЕЗ ПРИМЕРОВ, а то это перебор